Otonom Yazılım Geliştirme İçin Claude Code Mimarisi ve Konfigürasyon Standartları

1. Yönetici Özeti: Ajan Tabanlı Geliştirme Paradigması

Yazılım mühendisliği, basit metin tamamlama araçlarından durum bilgisine sahip (stateful) ve otonom işbirliği yapabilen ajanlara doğru evrilmektedir. Anthropic tarafından geliştirilen Claude Code, bu dönüşümün merkezinde yer almakta olup, yalnızca bir kod üreteci değil, dosya sistemleri üzerinde gezinebilen, terminal komutlarını yürütebilen ve karmaşık refaktör işlemlerini yönetebilen otonom bir mühendis olarak işlev görmektedir.[1] Bu paradigmatik değişim, geliştiriciler ve sistem mimarları için proje iskeletini oluşturma (scaffolding) süreçlerinde köklü değişiklikler gerektirmektedir. Nasıl ki insan mühendisler için işe alım dokümantasyonları (onboarding docs), mimari diyagramlar ve belirlenmiş izin yapıları hayati önem taşıyorsa, Claude Code için de yüksek güvenilirlik ve güvenlik standartlarında çalışabilmesi adına titizlikle tasarlanmış bir "Bilişsel Ortam" (Cognitive Environment) şarttır.[3]

Geleneksel geliştirme ortamlarından farklı olarak Claude Code, **Keşfet (Explore) → Planla (Plan) → Uygula (Act) → Doğrula (Verify)** döngüsü üzerinde çalışır.[5] Bu döngünün başarısı, ajanın erişebildiği bağlamın kalitesine ve yapılandırılmasına doğrudan bağlıdır. Yanlış yapılandırılmış bir proje hiyerarşisi veya eksik tanımlanmış bir CLAUDE.md dosyası, ajanın halüsinasyon görmesine, yanlış komutlar yürütmesine veya bağlam penceresini (context window) verimsiz kullanarak maliyetleri artırmasına neden olabilir.

Bu rapor, Claude Code ile otonom bir proje başlatmadan önce kurulması gereken teknik altyapıyı, dosya hiyerarşilerini, konfigürasyon önceliklerini ve genişletilebilirlik mekanizmalarını (Skills, Agents, Hooks, MCP) derinlemesine incelemektedir. Amaç, basit istem mühendisliğinin ötesine geçerek, kendi kendini denetleyebilen ve kurumsal standartlara uygun uzman seviyesinde ajan iş akışları oluşturmaktır.

---

2. Dosya Sistemi Hiyerarşisi ve Konfigürasyon Kapsamları

Deterministik ve iyi yapılandırılmış bir dosya sistemi, her Claude Code projesinin temelidir. Ajan, davranışlarını, yetkilerini ve kullanabileceği araçları belirlemek için katı bir konfigürasyon dosyası hiyerarşisine güvenir. Bu hiyerarşi, yerel (local), projeye özgü (project-specific) ve küresel kullanıcı (global user) ayarlarının nasıl birleştirileceğini belirleyen kesin bir öncelik sırasını takip eder. Bu yapıyı anlamak, bir aracın neden engellendiğini veya bir model

davranışının neden değiştirilemediğini teşhis etmek için kritik öneme sahiptir.

2.1 Konfigürasyon Öncelik Matrisi

Claude Code, ayarları birden fazla kaynaktan okuyarak nihai çalışma zamanı konfigürasyonunu oluşturur. Çakışma durumunda daha spesifik veya daha yüksek yetkili kapsamlar, daha genel olanları ezer.

Öncelik Sırası	Kapsam Türü	Dosya Konumu	Açıklama ve Kullanım Amacı
1. Yönetilen (Managed)	Sistem/Kurumsal	/Library/Application Support/ClaudeCode/managed-settings.json (macOS) /etc/claude-code/managed-settings.json (Linux)	**Değiştirilemez.** IT yöneticileri tarafından dağıtılan güvenlik politikalarıdır. Örneğin, dangerously-skip-permissions bayrağının kullanımını engelleyerek geliştiricilerin güvenlik duvarlarını aşmasını önler.[7]
2. CLI Argümanları	Çalışma Zamanı	Terminal Komutu (örn. --verbose)	O anki oturum için geçerli olan geçici geçersiz kılma (override) işlemleridir.
3. Yerel Proje	Kişisel/Geçici	.claude/settings.local.json	Geliştiricinin kendi makinesine özgü yollar veya hata ayıklama ayarlarıdır. Git tarafından yoksayılmalıdır (.gitignore).

4. Proje	Paylaşılan	.claude/settings.json	Projenin "tek gerçeklik kaynağıdır". Takım genelinde standartları, izin verilen araçları ve model sürümlerini belirler. Sürüm kontrolüne (Git) eklenmelidir.[7]
5. Kullanıcı	Küresel	~/.claude/settings.json	Geliştiricinin tüm projelerinde geçerli olan kişisel tercihleri (tema, bildirim ayarları, varsayılan model) içerir.[7]

2.2 Kök Dizin Standartları

Claude Code ile başlatılacak bir projenin kök dizini, ajanın kendini "konumlandırdığı" ve projenin sınırlarını anladığı noktadır. Aşağıdaki ağaç yapısı, otonom bir ajanın ihtiyaç duyduğu tüm meta verileri ve operasyonel talimatları içeren ideal bir hiyerarşiyi temsil etmektedir.

proje-koku/

├──.claude/ # Ajanın Beyni: Temel konfigürasyon dizini (ZORUNLU)

│ ├── settings.json # Takım genelinde paylaşılan proje ayarları

│ ├── settings.local.json # Yerel geçersiz kılmalar (.gitignore'da olmalı)

│ ├── mcp.json # Proje kapsamlı Model Context Protocol sunucuları

│ ├── agents/ # Özel alt ajan (subagent) tanımları

│ │ ├── code-reviewer.md

│ │ └── security-auditor.md

│ ├── skills/ # Projeye özgü yetenekler (Skills)

│ │ ├── database-migration/ # Yetenek klasörü (Skill Bundle)

```
│ │ ├── SKILL.md # Yetenek tanımı ve talimatları
│ │ └── schema-validator.py # Yetenek tarafından çalıştırılan script
│ │ └── api-integration/
│ │ └── SKILL.md
│ └── hooks/ # Yaşam döngüsü otomasyon scriptleri
│ ├── pre-commit-check.sh
│ └── hooks.json
├──.claude-plugin/ # (Opsiyonel) Eğer bir eklenti geliştiriliyorsa
│ └── plugin.json # Eklenti manifest dosyası
├── CLAUDE.md # Birincil proje bağlamı ve hafızası
├── AGENTS.md # Araçlar arası (Vendor-agnostic) talimat dosyası
├──.gitignore #.claude/settings.local.json buraya eklenmeli
└── README.md # İnsan odaklı dokümantasyon (Claude bunu da okur)
```

Bu yapı, Claude Code'un "Dosya Sistemi Tabanlı Keşif" (Filesystem-based Discovery) yeteneğinden tam olarak yararlanmasını sağlar. Ajan, .claude klasörünü tarayarak hangi yeteneklere (skills) ve araçlara sahip olduğunu otomatik olarak öğrenir.[9]

---

3. Bağlam Mühendisliği: CLAUDE.md ve Proje Hafızası

Yapay zeka ajanları için dokümantasyon hazırlamak, insanlar için hazırlamaktan temelden farklıdır. İnsanlar satır aralarını okuyabilir, eksik bilgileri deneyimleriyle tamamlayabilir. Ajanlar ise açık (explicit), yapılandırılmış ve yoğun bilgiye ihtiyaç duyar. CLAUDE.md, projenin "uzun süreli hafızası" veya "sistem istemi uzantısı" (system prompt extension) olarak işlev görür. Claude bir dizinde başlatıldığında, projenin amacını, mimarisini ve kodlama kurallarını anlamak için ilk olarak bu dosyayı okur.[3]

3.1 CLAUDE.md Anatomisi ve Tasarım İlkeleri

Etkili bir CLAUDE.md dosyası, **Aşamalı İfşa (Progressive Disclosure)** ilkesine göre tasarlanmalıdır. Tüm veritabanı şemasını veya API dokümantasyonunu bu dosyaya yığmak, ajanın bağlam penceresini gereksiz yere doldurur ve maliyeti artırır. Bunun yerine, bu dosya

ajana "neyin nerede olduğunu" ve "nasıl düşünmesi gerektiğini" anlatan bir harita olmalıdır.[4]

Önerilen CLAUDE.md Şablonu:

Proje Bağlamı: [Proje Adı]

Genel Bakış

- **Amaç**:
- **Mimari**:
- **Teknoloji Yığını**:

İnşa ve Çalıştırma Komutları

- **Kurulum**: npm install (Yarn kullanma!)
- **Geliştirme Sunucusu**: npm run dev (Port 3000'de çalışır)
- **Test**: npm test (Jest kullanır, entegrasyon testleri için Docker gerekir)
- **Lint**: npm run lint

Kodlama Standartları

- **Stil**: OOP yerine Fonksiyonel Programlama tercih edilir.
- **Formatlama**: 2 boşluk girinti, noktalı virgül kullanılmaz.
- **İsimlendirme**: Fonksiyonlar için camelCase, Bileşenler (Components) için PascalCase.
- **Hata Yönetimi**: Asla ham string fırlatma; özel AppError sınıfını kullan.

Dizin Yapısı

- /src/api: REST API denetleyicileri (Controllers).
- /src/core: İş mantığı ve domain varlıkları (Saf fonksiyonlar).
- /src/infra: Veritabanı adaptörleri ve dış servis istemcileri.

Kritik Kurallar

1. .github/workflows dizinini kullanıcıdan açık onay almadan değiştirme.
2. Bir düzeltme (fix) uygulamadan önce mutlaka /tests dizininde hatayı yeniden üreten bir test yaz (TDD).
3. Çıktı için logger modülünü kullan; console.log kesinlikle yasaktır.

Analitik Not: "İnşa ve Çalıştırma Komutları" bölümü hayati önem taşır. Bu bölüm olmadığında, Claude Code tıpkı bir stajyer gibi davranarak komutları tahmin etmeye çalışır (örneğin, proje npm kullanırken yarn denemesi yapabilir). Bu tahminler, "Keşif" (Explore) turlarının artmasına, hata mesajlarının analiz edilmesine ve dolayısıyla token maliyetlerinin yükselmesine neden olur.

Komutların açıkça belirtilmesi bu belirsizliği ortadan kaldırır.[5]

3.2 AGENTS.md: Araçlar Arası Birlikte Çalışabilirlik

CLAUDE.md Anthropic ekosistemine özgüyken, AGENTS.md dosyası yapay zeka kodlama araçları arasında (Cursor, GitHub Copilot, Windsurf vb.) ortak bir standart haline gelmektedir. Eğer projenizde farklı araçlar kullanan geliştiriciler varsa, kök dizinde bir AGENTS.md oluşturmak stratejik bir hamledir. Claude Code, CLAUDE.md bulunamadığı durumlarda bu dosyayı okuyacak veya mevcutsa ek bağlam olarak kullanacaktır.[13]

Strateji: CLAUDE.md dosyasını Claude Code'a özgü yetenekleri (Skills) ve komutları tanımlamak için, AGENTS.md dosyasını ise genel mimari kurallar için kullanın. Bu hibrit yaklaşım, takım içindeki araç çeşitliliğini destekler.

---

4. Yetenek Sistemi (Skills System) ve Genişletilebilirlik

Claude Code projelerinde "Yetenekler" (Skills), ajana statik bilgi vermenin ötesinde, ona yeni **iş akışları** öğretmek anlamına gelir. Bir Yetenek, belirli bir görevi (örneğin, veritabanı migrasyonu, PR incelemesi veya PDF analizi) yerine getirmek için gereken adım adım talimatların ve çalıştırılabilir kodların paketlenmiş halidir.[4]

4.1 Yetenek Klasör Yapısı ve SKILL.md

Yetenekler, .claude/skills/ dizini altında her biri kendi klasörüne sahip olacak şekilde yapılandırılır. Bu modüler yapı, ajanın yetenekleri birbirinden izole etmesini ve yalnızca ihtiyaç duyduğunda yüklemesini sağlar. Bir yetenek klasörü sadece talimatları değil, çalıştırılabilir scriptleri (Python, Bash) ve şablon dosyalarını da içerebilir.

Örnek Yapı:

.claude/skills/database-migration/

- SKILL.md: Yeteneğin beyni. Metadata ve talimatları içerir.
- migration-template.sql: SQL şablon dosyası.
- validate.py: Migrasyonun doğruluğunu kontrol eden Python betiği.

4.2 SKILL.md Ön ve Arka Planı

Her SKILL.md dosyası, iki ana bölümden oluşur: **YAML Frontmatter** (Meta Veri) ve **Markdown İçeriği** (Talimatlar).

Örnek: Veritabanı Migrasyon Yeteneği

---

name: db-migrate description: Prisma kullanarak yeni bir veritabanı migrasyonu oluşturur ve uygular. Kullanıcı "şemayı güncelle" veya "tablo ekle" dediğinde otomatik tetiklenir. allowed-tools: model: claude-3-5-sonnet-20241022

Veritabanı Migrasyon İş Akışı

Kullanıcı bir veritabanı değişikliği talep ettiğinde, aşağıdaki adımları sırasıyla uygula:

1. **Analiz**: İstenen yeni varlıkları veya alanları belirle.
2. **Şema Düzenleme**: prisma/schema.prisma dosyasını güncelle.
   - Veritabanı sütun isimleri için snake_case kullan.
   - Tablo isimlerinin çoğul olduğundan emin olmak için her modele @@map ekle.
3. **Migrasyon Üretimi**:
   - Komut: npx prisma migrate dev --name <açıklayıcı-isim>
4. **Doğrulama**:
   - Doğrulama betiğini çalıştır: python3.claude/skills/database-migration/validate.py
   - migrations/ klasöründe yeni SQL dosyasının oluştuğunu teyit et.

Hata Yönetimi

Eğer migrasyon başarısız olursa, schema.prisma değişikliklerini git checkout ile geri al ve hata logunu raporla.

4.3 Frontmatter Konfigürasyon Referansı

Frontmatter, yeteneğin ne zaman ve nasıl çalışacağını belirleyen kontrol mekanizmasıdır.[16]

Alan	Zorunluluk	Açıklama ve Stratejik Kullanım
name	Evet	Yeteneğin benzersiz kimliği. Slash komutu olarak kullanılır (örn. name: test -> /test). Sadece küçük harf ve tire içerebilir.
description	Evet	Router'ın yeteneği

		otomatik olarak ne zaman tetikleyeceğine karar vermesini sağlayan semantik açıklama. Açıklamanın kalitesi, ajanın yeteneği doğru zamanda kullanıp kullanmayacağını belirler.
disabled-model-invocation	Hayır	true olarak ayarlanırsa, yetenek sadece kullanıcı slash komutunu yazdığında çalışır. /deploy-prod gibi tehlikeli veya maliyetli işlemler için bu ayar mutlaka true yapılmalıdır.
allowed-tools	Hayır	Yeteneğin kullanabileceği araçların beyaz listesi (whitelist). Güvenlik için yeteneklerin erişimini kısıtlamak (Least Privilege) en iyi uygulamadır.
model	Hayır	Bu yetenek çalışırken kullanılacak modeli belirler. Basit metin işleme görevleri için claude-3-haiku seçilerek maliyet optimizasyonu yapılabilir.
context	Hayır	fork değeri verilirse, yetenek ana oturumdan izole edilmiş geçici bir alt ajan (subagent) bağlamında çalışır ve iş bitince bu bağlam silinir.

Derin İçgörü: description alanı sadece bir meta veri değildir; aslında ana ajanın yönlendiricisine (router) verilen bir "Prompt Injection"dır. "Kod yardımcısı" gibi muğlak bir açıklama, yeteneğin yanlış zamanlarda çalışmasına yol açar. "Kullanıcı bir React bileşenini

refaktör etmek istediğinde kullan" gibi spesifik bir açıklama, deterministik davranışı garanti eder.[4]

---

5. Otonomi Mimarisi: Alt Ajanlar (Subagents) ve Ajan Takımları

Yetenekler (Skills) "prosedürler" veya "reçeteler" iken, Alt Ajanlar (Subagents) "uzman personellerdir". Bir alt ajan, ana oturumdan bağımsız kendi sistem istemine (system prompt), araçlarına ve en önemlisi **kendi bağlam penceresine** sahip olan izole bir Claude örneğidir. Bu yapı, ana bağlamı kirletmeden derinlemesine araştırma yapılmasını sağlar.[17]

5.1 Alt Ajanlar vs. Yetenekler

Geliştiricilerin en sık yaptığı hata bu iki kavramı karıştırmaktır.

- **Yetenek Kullan:** Tekrarlanabilir, adımları belli bir süreç varsa (örn. "Bu JSON'ı formatla", "Testleri çalıştır"). Ana ajan bu adımları kendi hafızasında yürütür.
- **Alt Ajan Kullan:** Açık uçlu, keşif gerektiren bir problem varsa (örn. "Giriş sayfasının neden yavaş olduğunu araştır"). Ana ajan bu görevi "Performans Uzmanı" alt ajanına devreder. Alt ajan dosya sistemini tarar, profiler araçlarını çalıştırır, binlerce satır log okur ve ana ajana **sadece** "Sorun X satırındaki SQL sorgusunda" şeklinde özet bir rapor döner. Bu sayede ana ajanın bağlamı binlerce satırlık log ile dolmaz.[18]

5.2 Alt Ajan Konfigürasyonu

Alt ajanlar .claude/agents/ (proje bazlı) veya ~/.claude/agents/ (kullanıcı bazlı) dizininde tanımlanır.

Örnek: security-auditor.md

---

name: security-auditor description: Kod güvenliğini ve OWASP açıklarını inceleyen uzman ajan. model: claude-3-opus-20240229 tools: disallowedTools: color: purple

Sen bir Güvenlik Denetçisisin (Security Auditor). Amacın kod tabanındaki açıkları tespit etmektir.

1. SQL Injection, XSS ve kod içine gömülmüş (hardcoded) gizli anahtarlara öncelik ver.
2. Kodu ASLA düzeltme veya değiştirme (Edit yetkisi yok). Sadece açığın bulunduğu dosya

yolunu ve satır numarasını raporla.
3. Şüpheli gördüğün her durumu bildir, son derece muhafazakar davran.

Güvenlik Notu: disallowedTools alanı kritiktir. Bu alt ajanın sadece okuma (Read) yetkisi vardır, yazma (Edit) yetkisi yoktur. Bu "En Az Ayrıcalık İlkesi" (Principle of Least Privilege), ajanın otonom çalışırken sistemi bozmasını engeller ve ona güvenmenizi sağlar.[19]

5.3 Ajan Takımları ve tmux Orkestrasyonu (Deneysel)

Ajan Takımları (Agent Teams), hiyerarşik delegasyon yerine eşler arası (peer-to-peer) işbirliğine olanak tanıyan deneysel bir özelliktir. Bu modda, "Frontend Geliştirici", "Backend Geliştirici" ve "QA Mühendisi" gibi birden fazla ajan aynı anda çalışır ve birbirlerine mesaj göndererek koordine olurlar.[20]

Gereksinimler:

Bu özellik, terminal ekranını bölmek için tmux (terminal multiplexer) yazılımına ihtiyaç duyar. Her ajan tmux içinde ayrı bir panelde (pane) çalışır, böylece kullanıcı tüm ajanların düşünce süreçlerini eşzamanlı olarak izleyebilir.

Aktifleştirme:

Özellik deneysel olduğundan, ortam değişkeni ile açılmalıdır (settings.json içinde):

JSON

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Maliyet Uyarısı: Ajan takımları, aktif ajan sayısıyla doğru orantılı olarak token tüketir. 4 ajanlı bir takım, bağlam penceresini 4 kat daha hızlı tüketir. Bu özellik sadece karmaşık mimari değişiklikler veya çok modüllü özellik geliştirmeleri gibi insan koordinasyon maliyetinin token maliyetinden yüksek olduğu durumlarda kullanılmalıdır.[22]

---

6. Bağlanabilirlik: Model Context Protocol (MCP)

Varsayılan olarak Claude Code izole bir ortamda çalışır. Veritabanlarına, bulut altyapısına veya

hata takip sistemlerine (JIRA, Sentry) bağlanmak için **Model Context Protocol (MCP)** kullanılır. MCP, LLM'lerin dış dünya ile konuşmasını standartlaştıran bir protokoldür.[23]

6.1 .mcp.json ile Sunucu Tanımlama

Projeye özgü MCP sunucuları, kök dizindeki .mcp.json dosyasında tanımlanır. Bu dosya, ajanın hangi dış sistemlere erişebileceğini belirler.

Örnek Konfigürasyon:

JSON

```
{
  "mcpServers": {
    "postgres-primary": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-postgres",
        "postgresql://user:password@localhost:5432/mydb"
      ],
      "env": {
        "PGSSLMODE": "disable"
      }
    },
    "github-integration": {
      "command": "docker",
      "args":
    }
  }
}
```

6.2 Taşıma Katmanları (Transports) ve Güvenlik

MCP sunucuları farklı taşıma yöntemleri kullanabilir. Claude Code ile entegrasyonda en yaygın kullanılanlar şunlardır:

Taşıma Türü	Açıklama	Kullanım Senaryosu	Güvenlik Notu

stdio	Standart Girdi/Çıktı üzerinden iletişim kuran yerel işlemler.	Yerel veritabanları, dosya sistemi araçları.	Sunucu, Claude Code'u çalıştıran kullanıcının yetkileriyle çalışır.
http (SSE)	Uzak sunucularla Server-Sent Events üzerinden iletişim.	Kurumsal API'ler, bulut servisleri.	Yetkilendirme (Auth) başlıkları gerektirir.

İpucu: JavaScript ekosisteminde MCP sunucularını npx ile çalıştırmak yaygındır ancak bu, ana makinede Node.js bağımlılığı yaratır. Daha izole ve tekrar üretilebilir bir ortam için, konfigürasyonda docker komutlarını kullanmak en iyi uygulamadır (Best Practice). Bu sayede ajan, ana makineyi kirletmeden gerekli araçları konteyner içinde çalıştırabilir.[25]

---

7. Yönetişim ve Otomasyon: Kancalar (Hooks)

Hooks (Kancalar), projenizin "bağışıklık sistemidir". Ajanın belirli yaşam döngüsü olaylarında (lifecycle events) otomatik olarak tetiklenen kurallar tanımlamanızı sağlar. Bu kurallar, ajanın tehlikeli komutlar çalıştırmasını engelleyebilir, kod formatlamasını zorunlu kılabilir veya ajana her oturum başında hatırlatmalar yapabilir.[26]

7.1 Kanca Türleri ve Yaşam Döngüsü

Kancalar settings.json içinde hooks anahtarı altında veya .claude/hooks/ dizininde çalıştırılabilir betikler (Bash, Python) olarak tanımlanır.

Kritik Yaşam Döngüsü Olayları:

- SessionStart: Oturum başladığında veya devam ettirildiğinde çalışır. Ajana proje durumunu hatırlatmak için idealdir.
- PreToolUse: Bir araç (Tool) çalıştırılmadan **hemen önce** tetiklenir. **Güvenlik için en kritik kancadır.** İşlemi engelleyebilir (Block).
- PostToolUse: Araç çalıştıktan sonra tetiklenir. Otomatik linting veya loglama için kullanılır.

7.2 Örnek: "Gatekeeper" (Kapı Bekçisi) Güvenlik Kancası

Aşağıdaki örnek, Claude'un .env dosyalarını düzenlemesini veya okumasını engelleyen bir güvenlik kancasıdır. Bu, API anahtarlarının yanlışlıkla silinmesini veya sızdırılmasını önler.

settings.json Konfigürasyonu:

JSON

```
"hooks": {
  "PreToolUse":
}
```

Betik .claude/hooks/protect-secrets.sh:

Bash

```
#!/bin/bash
# Claude Code, olay verisini stdin üzerinden JSON olarak gönderir
INPUT=$(cat)
# jq aracı ile dosya yolunu ayıkla
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path')

# Dosya yolu.env içeriyorsa işlemi engelle
if]; then
  # stderr'e yazılan mesaj ajana hata nedeni olarak gösterilir
  echo "HATA:.env dosyalarına erişim güvenlik politikası gereği yasaklanmıştır. Lütfen kullanıcıdan manuel işlem talep et." >&2
  exit 2 # Çıkış kodu 2, eylemin BLOKE edildiğini belirtir
fi

exit 0 # Çıkış kodu 0, eyleme İZİN verir
```

Mekanizma Analizi: exit 2 çıkış kodu, Claude Code protokolünde özel bir anlama sahiptir. Bu kod, ajana "İsteğin reddedildi, nedeni stderr çıktısındadır, planını buna göre güncelle" mesajını verir. Bu sayede ajan hatayı anlar ve alternatif bir yol (örn. kullanıcıdan yardım istemek) dener.[12]

---

8. Dağıtım ve Paketleme: Eklenti (Plugins) Sistemi

Oluşturduğunuz bu karmaşık yapılandırmayı (Yetenekler, Ajanlar, Kancalar, MCP Sunucuları) tüm organizasyona dağıtmak için Claude Code'un Eklenti sistemi kullanılır. Bir eklenti, tüm bu bileşenleri içeren ve kök dizininde bir plugin.json manifest dosyası bulunan bir klasördür.[9]

8.1 plugin.json Manifest Yapısı

Eklenti dizininin .claude-plugin/ alt klasöründe bulunmalıdır.

JSON

```
{
  "name": "kurumsal-standart-paket",
  "version": "1.0.0",
  "description": "Acme Corp için standart linting yetenekleri ve güvenlik kancaları.",
  "commands":,
  "agents": [
    "agents/compliance-officer.md"
  ],
  "hooks": {
    "PreToolUse":
  }
}
```

Bu yapı, Platform Mühendisliği ekiplerinin bir "Standart Geliştirme Ortamı" eklentisi hazırlamasına olanak tanır. Geliştiriciler bu eklentiyi tek komutla kurarak, projelerinde manuel konfigürasyon yapmadan tüm güvenlik kurallarına ve özel yeteneklere sahip olurlar.

---

9. Sonuç ve En İyi Uygulama Tavsiyeleri

Claude Code ile başarılı, otonom ve güvenli bir proje ortamı oluşturmak için, yukarıdaki teknik detayların ışığında şu "Altın Kurallar"a uyulmalıdır:

1. **Tek Gerçeklik Kaynağı (One-Brain Principle):** Proje bağlamını parçalamayın. Tüm temel bilgileri CLAUDE.md içinde, araçtan bağımsız bilgileri AGENTS.md içinde tutun.
2. **Plan Modu Önceliği:** Varsayılan izin modunu plan olarak ayarlayın ("permissions": { "defaultMode": "plan" }). Bu, ajanın kod yazmadan önce size detaylı bir plan sunmasını zorunlu kılar ve "Tasarım Gözden Geçirme" sürecini simüle eder.[5]
3. **İzole Alt Ajanlar:** Alt ajanları tanımlarken disallowedTools özelliğini agresif bir şekilde kullanın. Bir dokümantasyon ajanının terminal erişimine ihtiyacı yoktur; bu erişimi kapatmak kazaları önler.
4. **Değişmez Yönetilen Ayarlar:** Kurumsal ortamda, geliştiricilerin güvenlik duvarlarını aşmasını önlemek için managed-settings.json dosyasını kullanın.

Bu raporda sunulan mimari, Claude Code'u basit bir CLI sohbet botundan, mühendislik ekibinizin tam entegre, otonom ve güvenilir bir üyesine dönüştürmek için gerekli tüm teknik

altyapıyı sağlamaktadır.